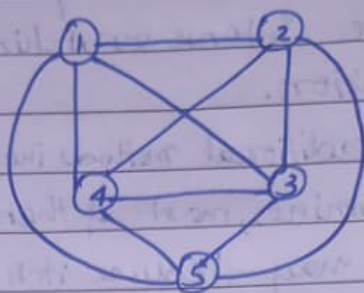


UNIT 4: Backtracking4.1 Travelling Salesman: Branch & Bound:# The given graph is:

Its cost adjacency matrix is given as:

	1	2	3	4	5
1	∞	20	30	10	11
2	15	∞	16	4	2
3	3	5	∞	2	4
4	19	6	18	∞	3
5	16	4	7	16	∞

Step 1: we need to reduce the matrix.

(i) finding the minimum value from each row
& subtracting it from whole row.(ii) then, finding min. value from each column
& subtracting it from whole column.

(iii) add up all the reduced values of both rows & columns to get the cost of reduction (r)

(i) Row reducing the given matrix:

	1	2	3	4	5	
1	∞	10	20	0	1	10
2	13	∞	14	2	0	2
3	1	2	∞	0	2	2
4	16	3	15	∞	0	3
5	12	0	3	12	∞	4
						<u>21.</u>

(ii) Column Reducing the given matrix:

	1	2	3	4	5
1	∞	10	10	0	1
2	12	∞	17	2	0
3	0	2	∞	0	2
4	15	3	12	∞	0
5	11	0	0	12	∞
	1	2	3	0	0

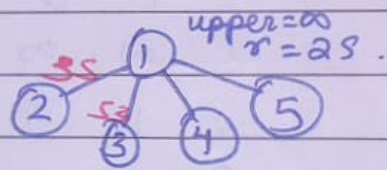
4.

So, the given matrix has been reduced.
and $\sigma = 21 + 4 = 25$. [minimum cost of tour may be 25].

Now, hence forth, we will be using the reduced matrix only:

We first make a STATE SPACE TREE:

① Starting with vertex 1:



Now, we need to find for

$1 \rightarrow 2, 1 \rightarrow 3, 1 \rightarrow 4, 1 \rightarrow 5$.

We do this by taking the reduced matrix, and then suppose for $1 \rightarrow 2$; we write ' ∞ ' for 1st row & 2nd column and also $[2, 1]$; $\because$ it cannot go back to 2.

similarly for all.

However, if it is not reduced, i.e., there is not even a single '0' in every row or column, it has to be reduced again.

P.T.O.

① for ①→②;

	1	2	3	4	5	
1	∞	∞	∞	∞	∞	∞
2	∞	∞	11	20	80	0
3	∞	∞	∞	0	2	0
4	15	∞	12	∞	0	0
5	11	∞	0	12	∞	0
	0	∞	0	0	0	0

∴ reduced.

∴ 1→2; cost: $c(1,2) + \tau + \hat{\tau}$
 $= 10 + 25 + 0 = 35.$

from original reduced matrix → reduced row.

② for ①→③;

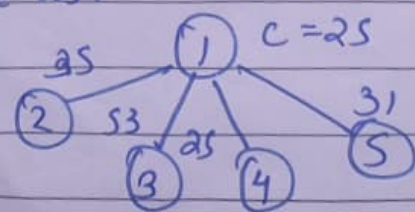
	1	2	3	4	5	
1	∞	∞	∞	∞	∞	∞
2	12	∞	∞	2	0	0
3	∞	2	∞	0	2	0
4	15	3	∞	∞	0	0
5	11	0	∞	12	∞	0
	?	0	∞	0	0	0

∴ $\hat{\tau} = 11$

∴ 1→③; cost: $c(1,3) + \tau + \hat{\tau}$
 $= 10 + 25 + 11$
 $= 46$ ~~53~~

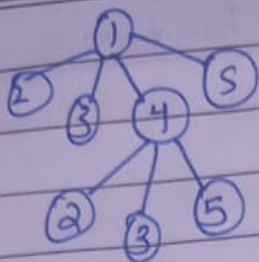
Similarly, we can do for 1→④ and 1→5.

∴ Results come as:



Here 1→4 is the lowest cost.

∴ that leaf is extended,



Now, we have to find for
 $4 \rightarrow 2$, $4 \rightarrow 3$, $4 \rightarrow 5$.

$\therefore$ We use the reduced matrix
 of when we found $1 \rightarrow 4$.

i.e. $\Rightarrow$

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	11	∞	0
3	0	3	∞	∞	2
4	3	12	∞	0	
5	11	0	0	∞	∞

Note: It should be reduced.

i.e., all rows/columns $\rightarrow \infty$.

either 1 value in it $\rightarrow 0$.

for $4 \rightarrow 2$, row 4th, 2nd column $\rightarrow \infty$.
 also, $[2, 4] \rightarrow \infty$.

like;

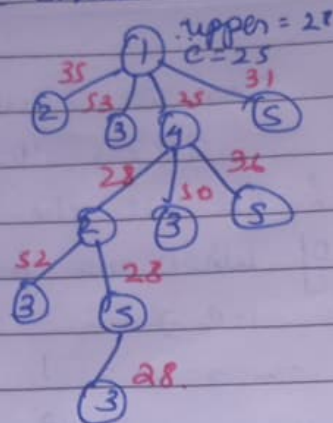
$4 \rightarrow 2$:

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	11	∞	0
3	0	∞	∞	∞	2
4	∞	∞	∞	∞	∞
5	11	∞	0	∞	∞

$\therefore$ Here, cost: $C(4, 2) + r + s$
 $= 3 + 25 + 0 = 28$.

$\therefore$ When we keep going, $4 \rightarrow 2$, $4 \rightarrow 3$, $4 \rightarrow 5$ is compared,
 and the least node is continued with

Finally, the STATE SPACE TREE is found as:

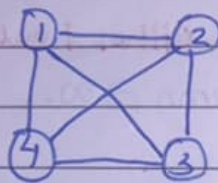


∴ The minimum cost = 28.

Time complexity: $O(n!)$

4.2 Travelling Salesman by Dynamic Programming

Given Graph:



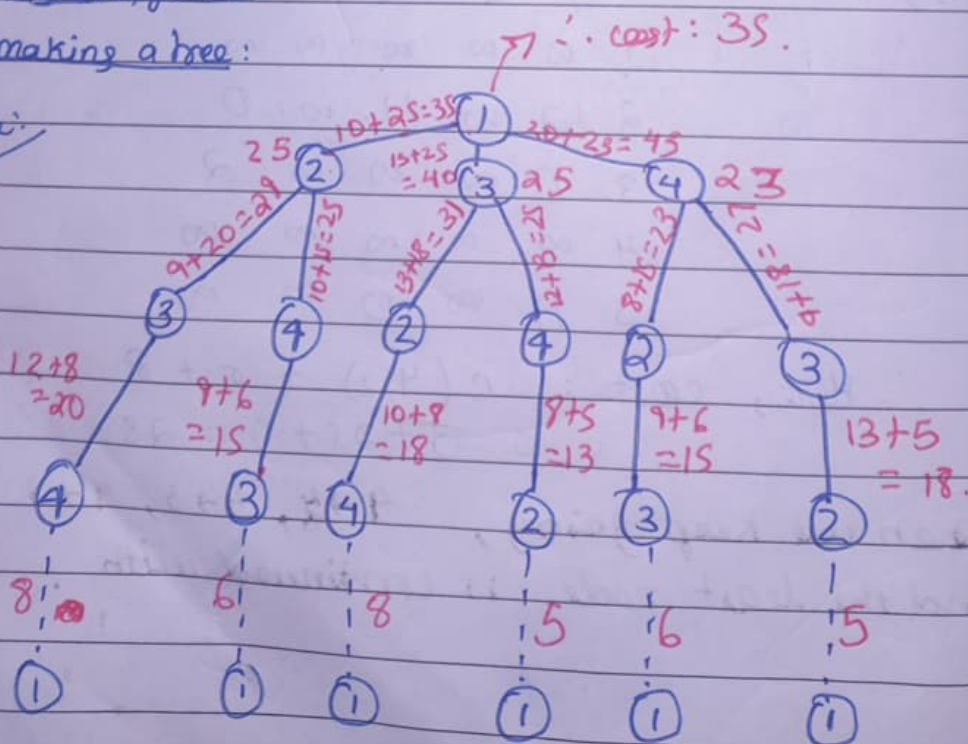
cost adjacency
diagram:

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Using DP's brut force.

Step 1: making a tree:

Brut force:



Now, we enter it by keeping the cost at each line.

○ → upper → row

○ → lower → column.

• We also add the given value at each level.

• If anywhere merges, we take the least value.

By DP: [All need cost to reach 1]

$$g(2, \emptyset) = 5 \quad \text{for easy: } g(2) = 5$$

$$g(3, \emptyset) = 6 \quad g(3) = 6$$

$$g(4, \emptyset) = 8 \quad g(4) = 8$$

Only taking best

$$g(2, \{3\}) = 15$$

$$g(2, \{3, 4\}) = 25$$

$$g(2, \{4\}) = 18$$

$$g(3, \{4, 2\}) = 25$$

$$g(3, \{2\}) = 18$$

$$g(4, \{2, 3\}) = 23$$

$$g(3, \{4\}) = 20$$

$$g(4, \{2\}) = 13$$

$$g(4, \{3\}) = 15$$

$$\therefore g(1, \{2, 3, 4\})$$

$$= \min (C_{12} + g(2, \{3, 4\}), C_{13} + g(3, \{4, 2\}), C_{14} + g(4, \{2, 3\}))$$

$$= 35$$

Time Complexity : $O(2^n \times n^2)$

4.3 Hamiltonian Cycle using Backtracking

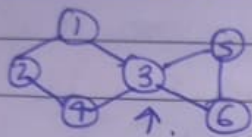
When a graph is given, we need to visit each vertex only once, and return.

Hence, we need to find all possible tours. Takes exponential time, causes NP Hard problem.

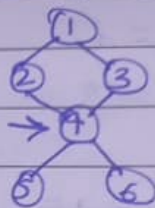
Hamiltonian Cycle $\rightarrow$ is a graph cycle (closed loop) through a graph that visits each node exactly once.

Hamiltonian Cycle cannot exist in:

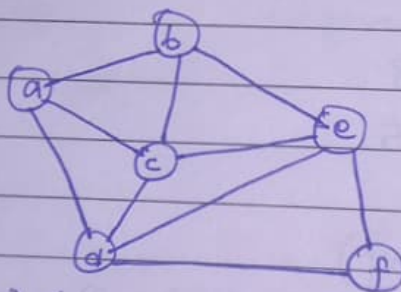
① Articulate Point:



② Pendant Vertices:



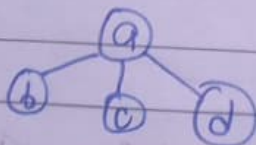
Q) Find Hamiltonian Cycle in:



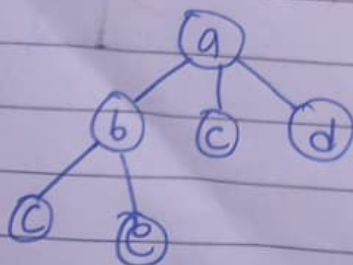
Step 1) Starting from 'a'.

State Space tree:

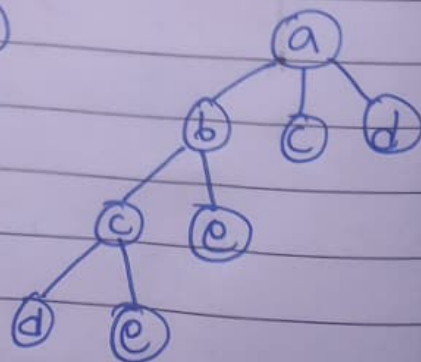
①

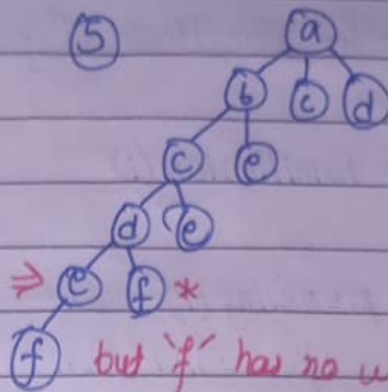
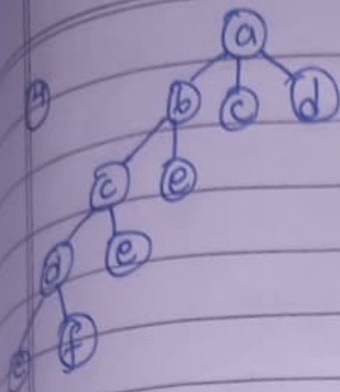


②

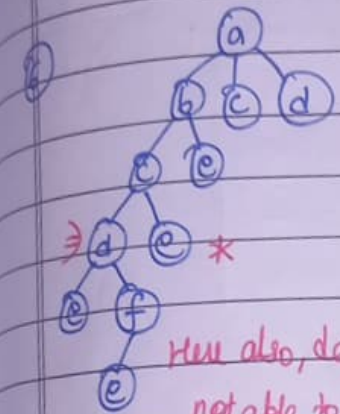


③

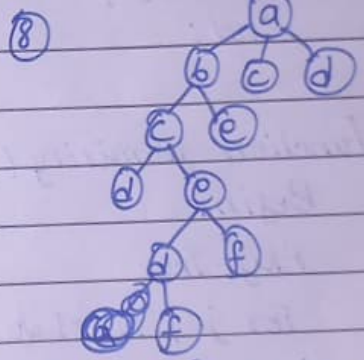
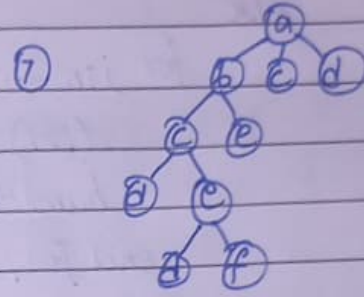




⇒ but 'f' has no way to reach 'a'.
 ∴ we backtrack, and try the adjacent node 'f' of 'e'.

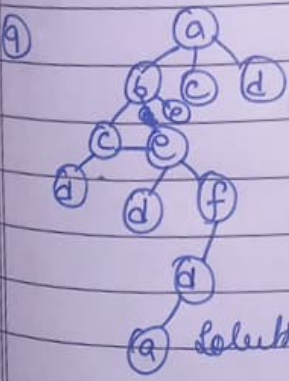


Here also, deadend, not able to reach 'a'.
 ∴ We now choose subtree 'e' adjacent to 'd'.



No Solution found (Deadend).

∴ Hamiltonian Cycle: ~~{a, b, c, d, e, f}~~ So now we choose adjacent 'f'.



∴ Hamiltonian Cycle:
~~{a, b, c, d, e, f}~~
 {a, b, c, e, f, d, a}

Solution found.

Time Complexity: $O(n!)$

Algorithm for Hamiltonian Cycle:

```

procedure hamiltonian(i)
begin
    if premising(i) then
        if  $i = n-1$  then
            display  $(v(0..n-1))$ 
        end if
    else
        for  $j$  in  $2..n$  loop
             $v(i+1) := j$ ;
            hamiltonian( $i+1$ );
        end for;
    end if;
End;

```

Function premising(i)

```

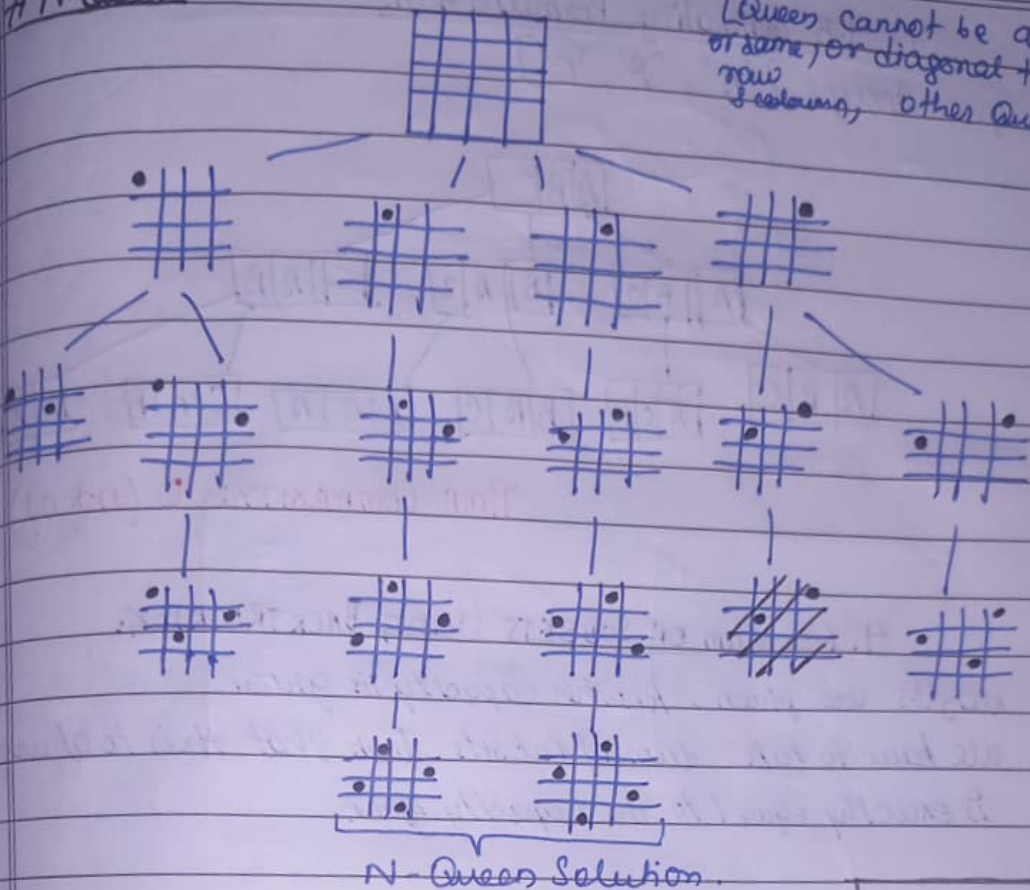
Begin
    Flag = True
    For  $j = 1$  to  $i-1$  do
        if (vertex  $v_i$  and  $v_j$  are neighbours)
            Flag = false
        end if
    End for
    If ( $v_i$  and  $v_{i-1}$  are neighbours)
        Flag = true
    End if
    Return (Flag)
End.

```

4.4 N-Queen

N-Queen:

[Queens cannot be adjacent, or same, or diagonal to another row & column, other Queens.]



$$T(n) = O(N^n)$$

~~$$O(N!)$$~~

ALGORITHM:

// Checking for cell is attacked or not.

int is_attack (int i, int j)

{ int k, l

// checking if there is a queen or not.

in row or column.

for (k=0; k<N; k++)

{ if ((board[i][k]==1) || (board[k][i]==1))

return 1; }

// checking for diagonals.

for (k=0; k<N; k++)

{ for (l=0; l<N; l++)

{ if ((k+l)==(i+j) || (k-l)==(i-j))

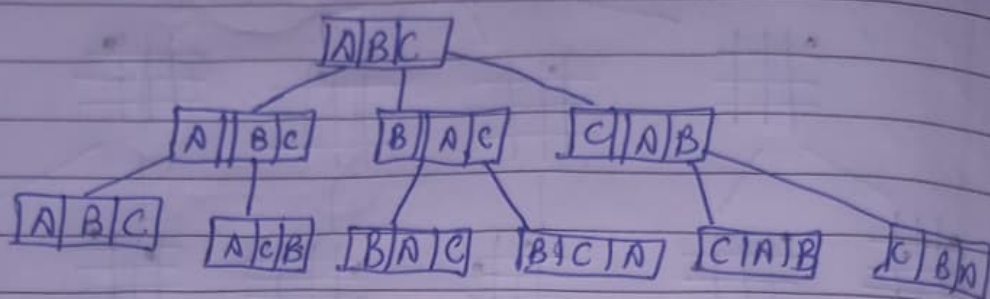
{ if (board[k][l]==1)

return 1; } }

return 0; }

4.5 Generating Permutation.

Given Array: ['A', 'B', 'C']

Time Complexity: $O(n \times n!)$ 4.6 Sum of Subsets Using Backtracking.

Weights are given. Another capacity is given.

We have to take sum of subsets such that their total weight is exactly equal to the capacity given.

Q) ~~m~~: $n=6$, ~~m~~ $= 30$.

	1	2	3	4	5	6	Total weight:
$w[1:6] = \{$	5	10	12	13	15	18	73
$\}$							

 $x =$

1	1	0	0	1	0
---	---	---	---	---	---

we can either take whole '1'

or none '0'.

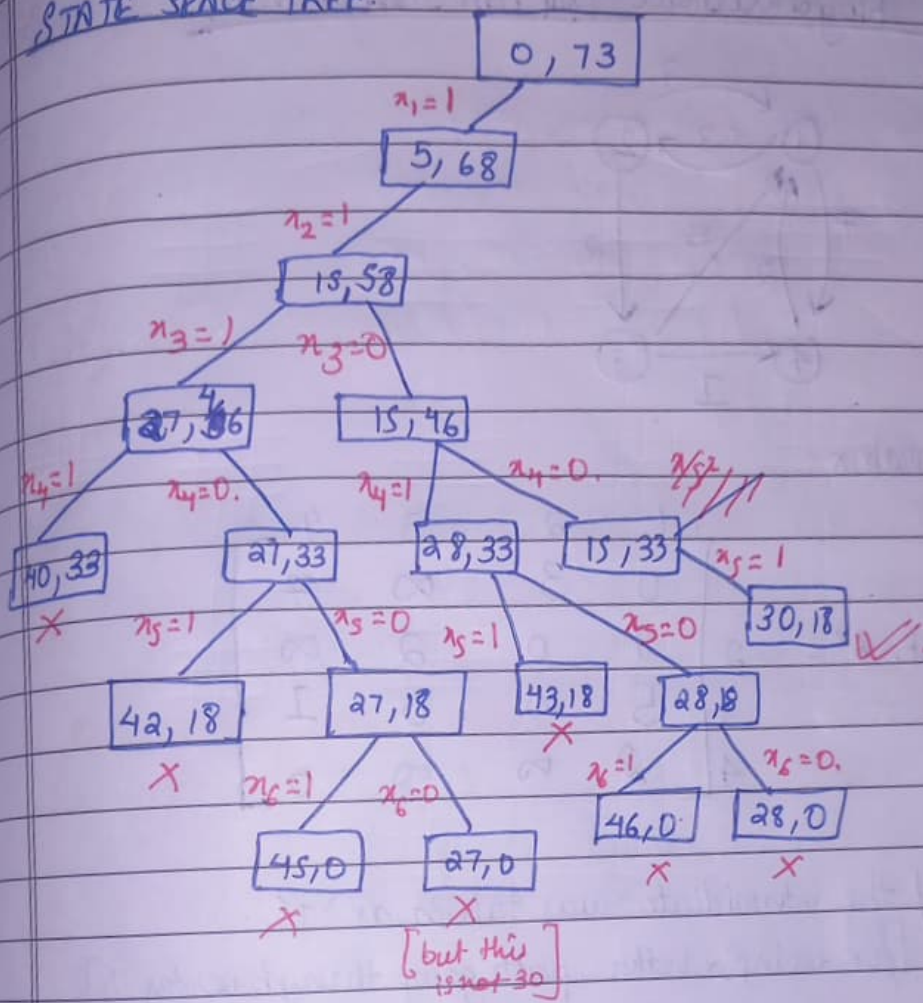
Here, sum of subsets has to be exactly equal to 30.

We use backtracking.

Time Complexity: $O(2^n)$

At the height root node, we write $[0, 73]$ ^{when weight = 30} ^{total weight}; then by choosing which object to choose at every step, we know when weight ^{weight = 30} it goes greater than 30, it is backtracked.

STATE SPACE TREE:

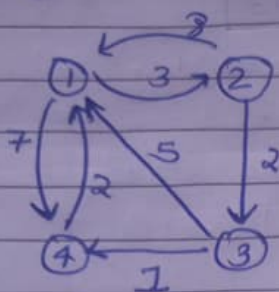


∴ One possible answer is: $\{1, 2, 5\}$.

Other possible answers: $\{3, 6\}$ and $\{1, 3, 5\}$.

4.6 Floyd Marshall (All Pair Shortest Path).

Given:



Making the matrix.

$$\text{Let } A^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

Now, 2) the intermediate was taken as '1':

[for seeing a better path going through vertex 1].

So,

Step 1) From previous matrix, all the paths ~~from~~ that belongs to vertex '1' should be left unchanged.

[i.e, 1st row, 1st column].

$$\text{matrix } A^1 = \begin{matrix} & \begin{matrix} 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & \bullet & \bullet \\ 5 & & 0 & \bullet \\ 2 & & & 0 \end{bmatrix} \end{matrix}$$

Aim: To find mixing values, but the least.

Original

Intermediate '1'

1. $A^0[2,3]$

$A^0[2,1] + A^0[1,3]$

2

<

8 + ∞

$\therefore A^1[2,3] = 2$

2. $A^0[2,4]$

$A^0[2,1] + A^0[1,4]$

∞

>

8 + 7

$\therefore A^1[2,4] = 15$

3. $A^0[3,2]$

$A^0[3,1] + A^0[1,2]$

∞

>

5 + 3

$\therefore A^1[3,2] = 8$

Similarly, we take all values for $A'[3,4]$, $A'[4,2]$, $A'[4,3]$.

$$\therefore A' = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{bmatrix} \end{matrix}$$

Now, Step 2: For intermediate '2':

from previous matrix, we define paths belonging to intermediate 2 unchanged. [i.e., 2nd row and 2nd column].

Note: Here, previous matrix is A' .

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & & \\ 8 & 0 & 2 & 15 \\ & 8 & 0 & \\ & 5 & & 0 \end{bmatrix} \end{matrix} \Rightarrow A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

original.

Intermediate '2'

$$\begin{array}{lcl} 1. & A'[1,3] & A'[1,2] + A'[2,3] \\ & \infty & 3 + 2 \quad \therefore A^2[1,3] = 5 \\ 2. & A'[1,4] & A'[1,2] + A'[2,4] \\ & 7 & 3 + 15 \quad \therefore A^2[1,4] = 7. \end{array}$$

Similarly, we find for all values.

⋮

Similarly we find A^3 by unchanging 3rd row & column

(from the A^2 matrix)

$$A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 4 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

And then find A^4 by unchanging the 4th row & column (from the A^3 matrix).

$$A^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 4 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{matrix}$$

Finally, All shortest paths $\Rightarrow$

TIME COMPLEXITY: $O(n^3)$

CODE:

```
for (K=1; K<=n; K++)  
{  
  for (i=1; i<=n; i++)  
  {  
    for (j=1; j<=n; j++)  
    {  
      A[i,j] = min(A[i,j], A[i,K] + A[K,j])  
    }  
  }  
}
```

4.7 0/1 KNAPSACK USING BRANCH AND BOUND.

~~Given:~~ Items:

The upper bound can be calculated with the formula:

$$UB = V + (m - w) \left(\frac{V_{i+1}}{W_{i+1}} \right)$$

where:

$w \rightarrow$ total weight of all objects placed in the knapsack.

$V \rightarrow$ total value (profit) of all objects placed in the knapsack.

$m \rightarrow$ capacity of the knapsack.

$\frac{V_{i+1}}{W_{i+1}} \rightarrow$ V/w of next object.

Q) Items:	A	B	C	D
weight:	7	5	4	3
profit:	42	25	40	12

①

Capacity of Knapsack = 10.

Solution $\Rightarrow$ we find the P/W (V/W) for all items.

Item A: $\frac{42}{7} = 6$, Item B: $\frac{25}{5} = 5$

Item C: $\frac{40}{4} = 10$, Item D: $\frac{12}{3} = 4$.

Now, we make table according to decreasing order of V/W .

Items	Value	Weight	V/W
1	40	4	10
2	42	7	6
3	25	5	5
4	12	3	4

Here, $m = 10$.

Calculations are as follows:

Step 1) Taking No item: $\therefore w = 0, v = 0, V/W = 10$
 $\therefore UB = v + (m - w) \left(\frac{V_{i+1}}{W_{i+1}} \right) = 0 + (10 - 0)(10) = 100$

Step 2) Calculation for taking item '1' and not taking item '1'.

Taking 1

$V = 40, W = 4, V/W = 6$

$\therefore UB = 40 + (10 - 4)(6)$
 $= 40 + 36 = 76$

Not taking 1

$V = 0, W = 0, V/W = 6$

$\therefore UB = 0 + (10 - 0)(6) = 60$

$\hookrightarrow$ max profit, $\therefore$ taking item '1'

Step 3) Calculation for taking item '2' and not taking item '2'.

Taking 2

$V = 42 + 40, W = 7 + 4, V/W = 5$
 $= 82, = 11$

Not taking 2

$V = 40, W = 4, V/W = 5$

$\therefore UB = 40 + (10 - 4)(5)$

$UB = 40 + 30 = 70$

but max $W = 10$;

$\therefore$ Not taking 2.

Not taking item '2' $\swarrow$

Step 4) Calculation for taking/not taking item '3'.

Taking 3

$V = 40 + 25, W = 4 + 5, V/W = 4$
 $= 65, = 9$

$\therefore UB = 65 + (10 - 9)(4)$
 $= 65 + 4 = 69$

$\hookrightarrow$ $\therefore$ taking item '3'.

Not taking 3

$V = 40, W = 4, V/W = 4$

$\therefore UB = 40 + (10 - 4)(4)$
 $= 40 + 24 = 64$

Steps) Calculation for taking and not taking item 4.

Taking item 4

$$V = 65 + 12; W = 9 + 3 \\ = 77; = 12$$

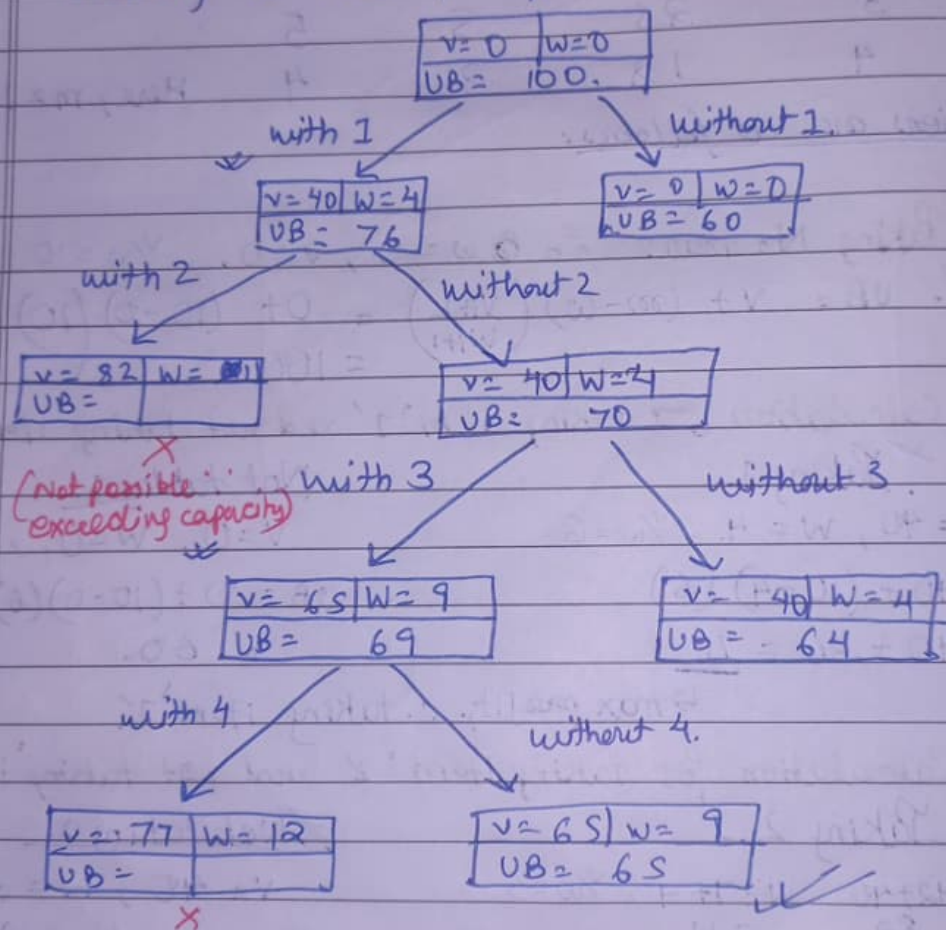
Not possible because
 $m = 10 < 12$

Not taking 4

$$V = 65; W = 9$$

$$UB = V + (m - W) \left(\frac{V_{i+1}}{W_{i+1}} \right) \\ = 65 + (10 - 9) (0) \\ = 65.$$

Making the Tree of Knapsack:



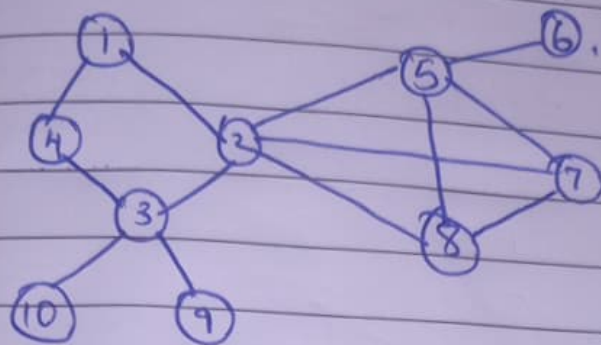
Ans: In this Knapsack, the maximum profit is 65.

Items took: {1, 3}

object set:

1 2 3 4
{1 0 1 0}

$$T(n) = O(2^n)$$

4.8 BFS & DFS:Given Graphs∴ By BFS:

1, 4, 2, 3, 5, 7, 8, 10, 9, 6.

or, 1, 2, 4, 5, 7, 8, 3, 6, 9, 10.

or, 5, 2, 8, 7, 6, 1, 3, 4, 10, 9, ~~10~~∴ DFS: 1, 2, 5, 6, 7, 8, 4, 3, 10, 9.

or, 1, 4, 3, 10, 9, 2, 8, 7, 5, 6.

or, 1, 2, 8, 7, 5, 6, 3, 10, 9, 4.

or, 3, 4, 1, 2, 5, 6, 7, 8, 10, 9.

Time Complexity of both: $O(V+E)$

✱